

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

BÁO CÁO HỆ NHÚNG

ĐỀ TÀI: THIẾT KẾ VÀ TRIỂN KHAI GAME 2048 TRÊN
VI ĐIỀU KHIỂN STM32F429

Giảng viên hướng dẫn: TS. Ngô Lam Trung

Sinh viên thực hiện: Thân Văn Long Nhật – 20215626

HÀ NỘI, 7/2026

Chương 1. Giới thiệu Dự án

Dự án tập trung vào việc nghiên cứu và xây dựng một trò chơi tương tác hoàn chỉnh – game 2048 – trên nền tảng hệ thống nhúng sử dụng vi điều khiển 32-bit ARM Cortex-M4 (STM32F429ZIT6). Trò chơi tích hợp màn hình màu LCD để hiển thị giao diện đồ họa chuyển động trực quan, sử dụng Joystick Analog ngoài để điều khiển các hướng vuốt số, còi chirp (Buzzer) phát tín hiệu âm thanh phản hồi sinh động và bộ nhớ Flash nội của vi điều khiển để lưu trữ điểm số kỷ lục (Best Score) bền vững.

Chương 2. Thiết kế và Cấu hình Phần cứng

2.1. Vi điều khiển trung tâm STM32F429ZIT6

Hệ thống sử dụng kit phát triển **STM32F429I-Discovery** tích hợp MCU STM32F429ZIT6 hiệu năng cao:

- Lõi ARM Cortex-M4 hoạt động ở tần số lên đến 180 MHz, tích hợp bộ xử lý số học dấu phẩy động (FPU).
- Bộ nhớ trong dung lượng lớn: 2 MB Flash phục vụ lưu trữ code chương trình và dữ liệu, 256 KB SRAM đáp ứng lưu trữ tạm thời các biến.
- Hỗ trợ bộ điều khiển bộ nhớ ngoài FMC (Flexible Memory Controller) và bộ điều khiển màn hình TFT LCD chuyên dụng (LTDC).

2.2. Các module ngoại vi tích hợp

2.2.1. Màn hình LCD TFT ILI9341 & SDRAM ngoại

Màn hình màu TFT LCD 2.4 inch sử dụng driver ILI9341 được kết nối với vi điều khiển thông qua giao diện song song RGB 16-bit (LTDC) giúp tăng tốc độ quét khung hình cực nhanh. Quá trình cấu hình thanh ghi và khởi tạo ban đầu cho LCD được truyền qua giao tiếp SPI5. Do bộ nhớ SRAM nội của chip không đủ chứa Framebuffer đồ họa kích thước lớn, bộ nhớ ngoài SDRAM dung lượng 64 Mbits (8 MBytes) giao tiếp qua FMC được cấu hình làm phân đoạn lưu trữ Framebuffer cho TouchGFX.

2.2.2. Bộ điều khiển Joystick Analog

Joystick analog cung cấp 2 ngõ ra điện áp tương tự tương ứng với trục X và Y. Hai ngõ ra

này được nối tới các kênh ADC của vi điều khiển (ADC1 và ADC2) cấu hình chế độ quét. Giá trị điện thế đọc về (0 đến 4095 tương ứng 0V - 3V) được phân tích để phát hiện thao tác điều khiển:

- **Vùng trung tâm (Center):** Điện áp nằm trong khoảng an toàn (giá trị ADC từ 1700 đến 2300).
- **Hướng Lên (Up - 'W'):** Trục dọc (X) giảm xuống dưới 1000 (đọc từ kênh ADC1_IN13).
- **Hướng Xuống (Down - 'S'):** Trục dọc (X) tăng vượt quá 3000 (đọc từ kênh ADC1_IN13).
- **Hướng Sang Phải (Right - 'D'):** Trục ngang (Y) giảm xuống dưới 1000 (đọc từ kênh ADC2_IN5).
- **Hướng Sang Trái (Left - 'A'):** Trục ngang (Y) tăng vượt quá 3000 (đọc từ kênh ADC2_IN5).

2.2.3. Còi chirp phát âm thanh (Buzzer)

Để tăng tính tương tác, còi chirp passive được kết nối với chân xuất tín hiệu PWM của bộ định thời TIM3 Channel 2. Bằng cách thay đổi tần số (Autoreload register - ARR) và chu kỳ nhiệm vụ (Compare register - CCR), hệ thống phát ra các tần số âm thanh khác nhau tạo hiệu ứng âm thanh di chuyển ô số và âm nhạc kết thúc game.

2.2.4. Bộ tạo số ngẫu nhiên phần cứng (Hardware RNG)

Khi chơi game 2048, việc chọn ngẫu nhiên vị trí ô trống để sinh ô mới và giá trị ô mới (2 hoặc 4) đòi hỏi tính ngẫu nhiên cao. Hệ thống sử dụng bộ tạo số ngẫu nhiên phần cứng (RNG) tích hợp sẵn trong STM32 để gieo hạt hoặc sinh trực tiếp các số ngẫu nhiên thay vì sử dụng thuật toán giả ngẫu nhiên phần mềm thông thường.

2.2.5. Bộ nhớ Flash nội (Sector 11)

Dữ liệu điểm số kỷ lục (Best Score) được ghi trực tiếp vào Sector cuối cùng của Flash nội (Sector 11, địa chỉ 0x080F0000). Mỗi khi người chơi đạt điểm số mới lớn hơn kỷ lục cũ, hệ thống sẽ thực hiện xóa Sector 11 và ghi đè giá trị Best Score mới vào địa chỉ này, giúp bảo lưu dữ liệu ngay cả khi kit bị tắt nguồn.

2.3. Sơ đồ kết nối chân ngoại vi (Pinout Connections)

Dưới đây là bảng cấu hình chi tiết sơ đồ kết nối chân giữa vi điều khiển STM32F429ZIT6 và các module ngoại vi bên ngoài được thiết lập trong CubeMX:

Thiết bị ngoại vi	Tên chân Module	Chân STM32	Chế độ cấu hình (Mode)	Ghi chú chức năng
Joystick Analog	VCC	3V / 3.3V	Power	Cấp nguồn dương
	GND	GND	Power	Cấp nguồn âm
	VRX (Trục dọc)	PC3	ADC1_IN13 (Analog Mode)	Đọc giá trị trục dọc (12-bit Resolution)
	VRY (Trục ngang)	PA5	ADC2_IN5 (Analog Mode)	Đọc giá trị trục ngang (12-bit Resolution)
Còi Buzzer (Passive)	Signal (Tín hiệu)	PA7	TIM3_CH2 (PWM Generation)	Phát xung PWM điều khiển tần số âm thanh
	GND	GND	Power	Cấp nguồn âm

Chương 3. Thiết kế và Hiện thực Phần mềm

3.1. Kiến trúc Đa nhiệm FreeRTOS

Hệ thống phần mềm được tổ chức chạy trên hệ điều hành thời gian thực FreeRTOS, giúp

tách biệt các luồng xử lý và đảm bảo tính phản hồi nhanh của giao diện. Có hai tác vụ (tasks) hoạt động song song:

1. **Tác vụ Đọc Joystick** (`defaultTask`)
Chạy chu kỳ mỗi 100ms. Thực hiện khởi động ADC, đọc giá trị điện áp Joystick từ ADC1 và ADC2, lọc nhiễu dội, phân tích hướng đi ('W', 'S', 'A', 'D'). Nếu phát hiện hướng đi mới, nó sẽ gửi ký tự đại diện hướng đi vào Message Queue `myQueue01Handle`
2. **Tác vụ Giao diện** (`touchgfxTask`)
Xử lý vòng lặp đồ họa TouchGFX, vẽ màn hình và cập nhật lưới tọa độ.

3.2. Cấu trúc mã nguồn của Logic Game (Screen2View)

Màn hình trò chơi chính được cài đặt trong lớp `Screen2View` theo kiến trúc MVP của TouchGFX. Dưới đây là các hàm chức năng chính được hiện thực:

3.2.1. Đọc Queue Joystick và Xử lý ngắt Tick (`tickEvent`)

Mỗi chu kỳ quét màn hình (Tick event), TouchGFX gọi hàm `tickEvent()` Tại đây, luồng UI sẽ kiểm tra hàng đợi tin nhắn:

```
void Screen2View::tickEvent()
{
    uint8_t res;

    if (osMessageQueueGetCount(myQueue01Handle) > 0)
    {
        osMessageQueueGet(myQueue01Handle, &res, NULL, osWaitForever);

        bool boardChanged = false;

        switch (res)
        {
            case 'W': boardChanged = slideUp(); break;
```

```

        case 'S': boardChanged = slideDown(); break;

        case 'A': boardChanged = slideLeft(); break;

        case 'D': boardChanged = slideRight(); break;

    }

    if (boardChanged)
    {
        playSound(300, 30); // Phát âm thanh di chuyển ngắn

        addNewTile(); // Sinh thêm ô số mới

        updateBoard(); // Vẽ lại bảng số

        updateScore(); // Cập nhật điểm số hiện tại
    }

    if (isGameOver())
    {
        container_game_over.setVisible(true);

        container_game_over.invalidate();

        updateBestScore();

        playGameOverSound();
    }
}
}

```

3.2.2. Thuật toán dịch chuyển và gộp ô số (slide)

Cài đặt logic gộp ô số trong game 2048. Ví dụ hàm `slideLeft()` thực hiện dồn tất cả các số sang bên trái, gộp các ô cạnh nhau có giá trị bằng nhau và tính điểm cộng dồn:

```
bool Screen2View::slideLeft()

{

    bool moved = false;

    for (int i = 0; i < 4; i++)

    {

        int newPos = 0;

        int lastMerged = -1;

        for (int j = 0; j < 4; j++)

        {

            if (gameBoard[i][j] == 0) continue;


            if (newPos > 0 && gameBoard[i][newPos - 1] == gameBoard[i][j] && lastMerged != new
1)

            {

                gameBoard[i][newPos - 1] *= 2;    // Nhân đôi giá trị ô số

                score += gameBoard[i][newPos - 1]; // Cộng vào điểm số

                gameBoard[i][j] = 0;

                lastMerged = newPos - 1;

                moved = true;

            }

            else

            {

                if (newPos != j)

                {

                    gameBoard[i][newPos] = gameBoard[i][j];
```

```

        gameBoard[i][j] = 0;

        moved = true;

    }

    newPos++;

}

}

}

return moved;
}

```

3.2.3. Đọc/Ghi Best Score vào bộ nhớ Flash nội

Khi điểm số của lượt chơi hiện tại vượt qua điểm số kỷ lục cũ, hệ thống mở khóa Flash, thực hiện xóa sector 11 và ghi giá trị mới xuống:

```

void Screen2View::saveBestScoreToFlash()
{
    HAL_FLASH_Unlock();

    FLASH_EraseInitTypeDef eraseInit;
    uint32_t sectorError;

    eraseInit.TypeErase = FLASH_TYPEERASE_SECTORS;
    eraseInit.Sector = FLASH_SECTOR_11; // Địa chỉ 0x080F0000
    eraseInit.NbSectors = 1;
    eraseInit.VoltageRange = FLASH_VOLTAGE_RANGE_3;

    if (HAL_FLASHEx_Erase(&eraseInit, &sectorError) == HAL_OK)

```

```
{  
  
    HAL_FLASH_Program(FLASH_TYPEPROGRAM_WORD, FLASH_ADDR, bestScore);  
  
}  
  
HAL_FLASH_Lock();  
  
}
```

Chương 4. Kết quả Thực nghiệm và Giao diện Thực tế

Sau khi biên dịch và nạp thành công chương trình xuống Kit phát triển STM32F429I-Discovery, nhóm đã thực nghiệm chạy thử trò chơi. Kết quả thực tế cho thấy hệ thống hoạt động ổn định, mượt mà và đáp ứng đúng yêu cầu thiết kế ban đầu.

4.1. Giao diện khởi đầu trò chơi (Start Screen)

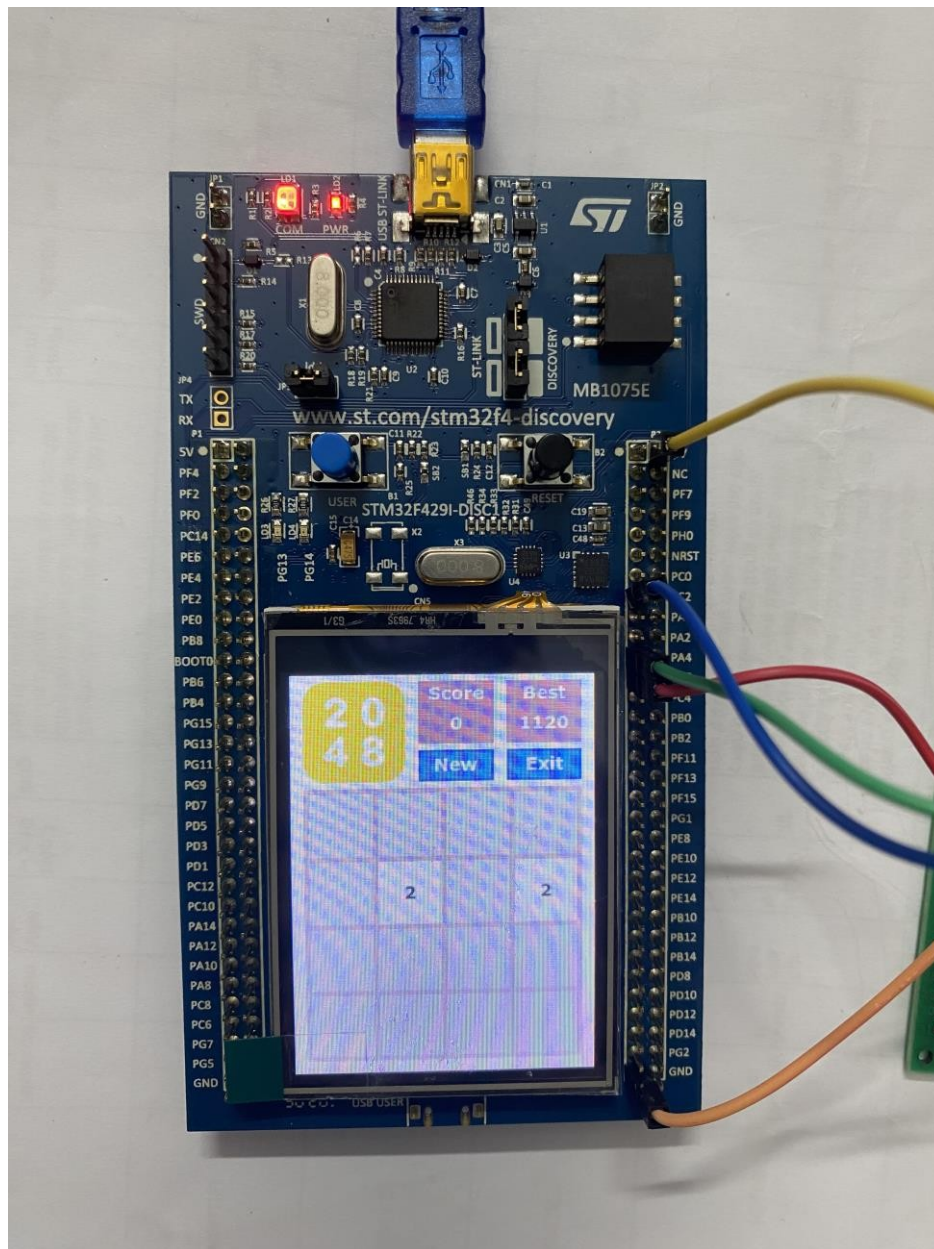
Khi hệ thống được cấp nguồn hoặc nhấn nút Reset, màn hình sẽ hiển thị logo trò chơi "2048" lớn màu vàng và nút "Play game". Người dùng có thể nhấn nút "Play game" để bắt đầu lượt chơi mới.



Hình 4.1. Giao diện màn hình khởi đầu (Start Screen) chạy thực tế trên Kit

4.2. Giao diện chơi game chính (Gameplay Screen)

Trong quá trình chơi, giao diện hiển thị bảng lưới 4x4 chứa các ô số, kèm theo bảng điểm hiện tại (Score) và điểm số kỷ lục (Best) được đọc từ bộ nhớ Flash. Khi sử dụng Joystick để vuốt di chuyển các ô số, còi buzzer sẽ phát tiếng "bíp" ngắn phản hồi.



Hình 4.2. Giao diện lưới ô số và điểm kỷ lục trong quá trình chơi game

4.3. Trạng thái kết thúc game (Game Over Screen)

Khi không còn bất kỳ nước đi hợp lệ nào (tất cả các ô đã đầy và các ô cạnh nhau không cùng giá trị), giao diện "Game over!" sẽ hiển thị dạng đè (overlay) màu xanh ngọc bán trong suốt, đồng thời còi buzzer phát âm thanh kết thúc game. Điểm số kỷ lục (Best Score) sẽ tự động cập nhật và ghi đè lưu trữ vào bộ nhớ Flash nếu điểm số lượt chơi này lớn hơn điểm cũ. Người chơi có thể nhấn nút "New Game" để bắt đầu lại hoặc "Exit" để quay lại màn hình chính.



Hình 4.3. Giao diện Game over hiển thị đề lên màn hình khi kết thúc lượt chơi

Chương 5. Hướng dẫn Biên dịch và Cài đặt

5.1. Môi trường phát triển

Dự án có thể mở trực tiếp bằng phần mềm **STM32CubeIDE** (bằng cách import thư mục **STM32CubeIDE**) để lập trình, debug và nạp code trực tiếp xuống kit qua mạch nạp ST-LINK tích hợp trên board.

5.2. Biên dịch thông qua dòng lệnh và Docker

Dự án hỗ trợ biên dịch hoàn toàn tự động bằng công cụ GNU Make và ARM GCC. Để biên dịch mà không cần cấu hình phức tạp trên máy chủ, dự án cung cấp giải pháp Docker đóng gói môi trường chuẩn (Ubuntu 22.04 + ARM GCC Toolchain).

Lệnh thực thi biên dịch dự án:

```
# Di chuyển vào thư mục dự án

cd stm32_game_2048

# Chạy biên dịch qua Make

make -C gcc -j8
```

Kết quả biên dịch thành công sẽ tạo ra file thực thi nhị phân `TouchGFX/build/bin/target.elf` và file nạp `intflash.hex` dùng để nạp xuống bộ nhớ Flash nội của vi điều khiển thông qua công cụ STM32CubeProgrammer.

Chương 6. Kết luận

Dự án đã hoàn thành thiết kế và xây dựng thành công game 2048 chạy mượt mà trên nền tảng nhúng STM32F429. Qua dự án, sinh viên đã nắm vững quy trình và cách triển khai, phát triển ứng dụng thời gian thực với FreeRTOS, lập trình đồ họa tương tác cao với TouchGFX dưới mô hình MVP, và các kỹ thuật can thiệp phần cứng nâng cao như PWM phát nhạc, lưu trữ dữ liệu bền vững trên bộ nhớ Flash nội, giao tiếp đa nhiệm hàng đợi và đọc chuyển đổi dữ liệu ADC từ cảm biến tương tự.